

SmartBIS Node watcher 구축 및 PostgreSQL 연동 기록 v1.0

이 문서는 `upload-store`를 감시하는 Node watcher 컨테이너를 만들고, 검증된 `contents.json`을 PostgreSQL에 저장하기까지의 과정과 Windows Git Bash에서 발생한 문제 및 해결 방법을 기록한다.

1. 최종 처리 흐름

```
upload-store/YYYY-MM-DD/contents.json
    ↓
Chokidar 파일 감시
    ↓
JSON 및 필수 필드 검증
    ↓
PostgreSQL INSERT/UPDATE
```

Node watcher는 외부 공개 API가 아니다. Compose 내부에서 `postgres-db:5432`로 통신하며 외부 포트는 공개하지 않는다.

2. 파일 구조

```
SmartBIS_jangsung/
├── server/node-watcher/
│   ├── Dockerfile
│   ├── package.json
│   └── src/index.js
├── server/db/init/001_create_content_table.sql
├── upload-store/YYYY-MM-DD/contents.json
└── compose.yaml
```

3. 기본 파일

`package.json`은 `chokidar`와 `pg`를 사용한다. JSON 파일에는 `//` 주석을 넣지 않는다.

```
{
  "name": "smartbis-node-watcher",
  "version": "1.0.0",
  "private": true,
  "main": "src/index.js",
  "scripts": {
    "start": "node src/index.js"
  },
  "dependencies": {
    "chokidar": "^4.0.3",
```

```
"pg": "^8.13.1"
}
```

Dockerfile:

```
FROM docker.io/library/node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY src ./src
RUN mkdir -p /app/upload-store
CMD ["npm", "start"]
```

4. Compose 서비스

```
node-watcher:
  build:
    context: ./server/node-watcher
  env_file:
    - .env
  environment:
    DB_HOST: postgres-db
    DB_PORT: 5432
    UPLOAD_ROOT: /app/upload-store
  volumes:
    - ./upload-store:/app/upload-store
  depends_on:
    - postgres-db
  restart: unless-stopped
```

컨테이너 간 DB 주소는 `localhost`가 아니라 Compose 서비스명인 `postgres-db`를 사용한다.

5. 데이터 계약

필수 필드:

```
content_id
content_type
title
content
template
```

NULL 허용 필드:

```
target_file_name
display_start_date
display_end_date
target_regions
```

summary는 현재 데이터 계약에서 사용하지 않는다.

6. Windows bind mount 문제

초기에는 Chokidar의 일반 파일 이벤트를 사용했다. 컨테이너 내부에는 파일이 보였지만 Windows 호스트에서 파일을 생성하거나 수정해도 검증 로그가 나오지 않았다.

이는 파일이 없거나 볼륨이 끊긴 것이 아니라 Windows/WSL bind mount에서 native 파일 이벤트가 안정적으로 전달 되지 않은 문제였다.

7. polling 방식으로 변경

Windows 공유 폴더에서는 다음 설정을 사용한다.

```
const watcher = chokidar.watch(UPLOAD_ROOT, {
  ignoreInitial: false,
  persistent: true,
  usePolling: true,
  interval: 500,
  awaitWriteFinish: {
    stabilityThreshold: 2000,
    pollInterval: 100
  }
});
```

설명:

usePolling: true	주기적으로 파일 상태 확인
interval: 500	500ms마다 변경 확인
stabilityThreshold: 2000	파일 쓰기가 2초간 안정된 후 처리

Polling은 native 이벤트보다 효율은 낮지만 Windows bind mount에서 업로드 완료와 파일 변경을 안정적으로 감지한다.

8. add와 change 이벤트 처리

새 파일은 **add**, 기존 **contents.json** 수정은 **change** 이벤트를 발생시킨다. 처음에는 **add**만 처리했기 때문에 기존 파일을 덮어쓸 때 검증되지 않았다.

최종 구현은 두 이벤트를 모두 처리한다.

```
const handleFile = async (filePath) => {  
  if (path.basename(filePath) === 'contents.json') {  
    await processContentsFile(filePath);  
  }  
};  
  
watcher.on('add', handleFile);  
watcher.on('change', handleFile);
```

컨테이너 시작 시 **add** 로그가 나오고, 파일 수정 후 **change** 로그가 다시 나오는 것은 정상이다.

9. PostgreSQL 테이블

```
CREATE TABLE IF NOT EXISTS contents (  
  content_id VARCHAR(100) PRIMARY KEY,  
  content_type VARCHAR(20) NOT NULL,  
  title TEXT NOT NULL,  
  content TEXT NOT NULL,  
  target_file_name VARCHAR(255),  
  display_start_date DATE,  
  display_end_date DATE,  
  target_regions JSONB,  
  template VARCHAR(100) NOT NULL,  
  source_path TEXT,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT contents_type_check  
    CHECK (content_type IN ('VIDEO', 'IMAGE', 'CARD')),  
  CONSTRAINT contents_date_check  
    CHECK (  
      display_start_date IS NULL  
      OR display_end_date IS NULL  
      OR display_end_date >= display_start_date  
    )  
);
```

10. 이미지 빌드와 실행

```
cd /d/PROJECT/SmartBIS_jangsung  
podman compose build node-watcher  
podman compose up -d node-watcher  
podman compose ps  
podman compose logs --tail=100 node-watcher
```

정상 시작 로그:

```
[watcher] 수신 폴더 확인: /app/upload-store/2026-09-24  
[database] 연결 성공: smartbis_db / smartbis_user  
[watcher] 감시 시작: /app/upload-store
```

소스 변경 후에는 반드시 다시 빌드하고 재생성한다.

```
podman compose build node-watcher  
podman compose up -d --force-recreate node-watcher
```

11. 정상 콘텐츠 테스트

```
{  
  "content_id": "TEST_20260924_0001",  
  "content_type": "VIDEO",  
  "title": "SmartBIS 테스트 영상",  
  "content": "Node watcher 연동 테스트용 콘텐츠입니다.",  
  "target_file_name": "test-video.mp4",  
  "display_start_date": "2026-09-24",  
  "display_end_date": "2026-09-30",  
  "target_regions": [  
    "장성읍"  
  ],  
  "template": "공지"  
}
```

정상 로그:

```
[content] 검증 성공: TEST_20260924_0001 (VIDEO)  
[database] 저장 성공: TEST_20260924_0001
```

12. PostgreSQL 저장 검증

```
MSYS_NO_PATHCONV=1 podman exec smartbis_jangsung-postgres-db-1 psql -U  
smartbis_user -d smartbis_db -c "SELECT content_id, content_type, title,  
target_file_name, display_start_date, display_end_date, target_regions, template,  
source_path FROM contents;"
```

정상 결과에는 다음 값이 포함된다.

```
TEST_20260924_0001 | VIDEO | SmartBIS 테스트 영상 | test-video.mp4 | 2026-09-24 |
```

2026-09-30 | ["장성읍"] | 공지

저장은 `content_id` 기준 UPSERT이므로 같은 콘텐츠가 다시 들어오면 중복 INSERT 오류 대신 기존 데이터가 갱신된다.

13. 잘못된 콘텐츠 테스트

`template`가 빠진 JSON은 거부되어야 한다.

```
[content] 처리 실패
[content] 사유: template 필드가 없습니다.
```

`summary`는 현재 계약에서 사용하지 않으므로 VIDEO라도 `summary`가 없어야 정상이다.

14. Git Bash 경로 변환 문제

다음 오류가 발생할 수 있다.

```
find: C:/Program Files/Git/app/upload-store: No such file or directory
```

Git Bash가 `/app/upload-store`를 Windows 경로로 바꾼 것이다. 컨테이너 내부 경로를 사용할 때 다음처럼 `MSYS_NO_PATHCONV=1`을 붙인다.

```
MSYS_NO_PATHCONV=1 podman exec smartbis_jangsung-node-watcher-1 ls -la /app/upload-store
```

15. 완료 기준

```
Node watcher 이미지 빌드 성공
Node watcher 컨테이너 Up
PostgreSQL 연결 성공
upload-store 감시 시작
Windows bind mount 파일 감지 성공
add 이벤트 처리 성공
change 이벤트 처리 성공
잘못된 JSON 거부
정상 JSON 검증 성공
contents 테이블 INSERT/UPDATE 성공
```